

AWS Data Pipeline Documentation

Amazon Reviews Dataset — S3 → Glue → Athena → Power BI

Project Overview

The objective of this project was to access the Amazon Reviews dataset stored in a teammate's AWS S3 bucket, create a metadata catalog using AWS Glue, query the data using Amazon Athena, and prepare it for Power BI visualization — without copying the data into my own AWS account.

Technologies Used

- Amazon S3
- AWS IAM
- AWS Glue
- AWS Glue Crawler
- AWS Glue Data Catalog
- Amazon Athena
- SQL
- Power BI

Architecture

The pipeline moves data from a shared S3 bucket through AWS Glue for cataloging, into Athena for querying, and finally into Power BI for visualization — with no data duplication at any stage.

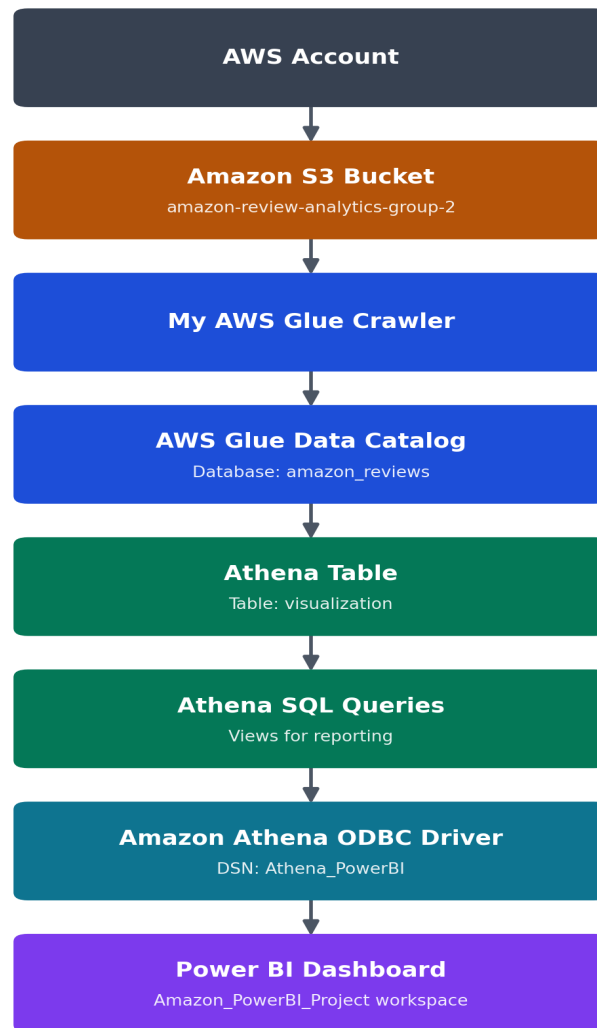


Figure 1. End-to-end data pipeline architecture.

Step 1: Access the Shared S3 Bucket

The Amazon Reviews dataset was stored in my teammate Afaan's AWS S3 bucket.

Bucket Name

```
amazon-review-analytics-group-2
```

Instead of copying the data into my own AWS account, I was granted the necessary IAM permissions to access the bucket directly. This approach avoids data duplication and ensures that the latest data is always available.

Step 2: Create an AWS Glue Crawler

In my AWS account, I opened the AWS Glue service and created a new crawler. The crawler configuration included:

- **Data Source:** Amazon S3
- **S3 Path:** The location of the dataset in Afaan's S3 bucket
- **IAM Role:** A role with permission to access the shared S3 bucket
- **Target Database:** amazon_reviews

The Glue crawler scans the files stored in Amazon S3 and automatically detects the schema of the Parquet files.

Step 3: Create the Glue Database

During crawler configuration, I created a new AWS Glue database named:

```
amazon_reviews
```

The database acts as a metadata repository where all discovered tables are stored. No actual data is copied into the database; only the metadata (schema, column names, data types, and file locations) is stored.

Step 4: Run the Glue Crawler

After configuring the crawler, I executed it. The crawler performed the following tasks:

- Connected to the shared S3 bucket.
- Scanned all Parquet files.
- Detected the schema automatically.
- Identified columns and their data types.
- Created the required metadata tables inside the AWS Glue Data Catalog.

Once the crawler completed successfully, the database contained the required table metadata.

Step 5: Verify the Tables

After the crawler finished, I verified that the required table had been created successfully.

Database	amazon_reviews
Table	visualization

The table contains metadata that points directly to the Parquet files stored in the shared S3 bucket.

Step 6: Query the Data Using Amazon Athena

Next, I opened Amazon Athena and selected:

- **Data Source:** AwsDataCatalog
- **Database:** amazon_reviews

I verified that the crawler had created the table correctly by running the following SQL query:

```
SELECT *
FROM "AwsDataCatalog"."amazon_reviews"."visualization"
LIMIT 10;
```

The successful execution of this query confirmed that Athena could read the data directly from the shared S3 bucket using the metadata generated by AWS Glue.

Step 7: Validate the Dataset

To ensure that all product categories were available, I executed additional SQL queries to inspect sample records from each category. This validation confirmed that the dataset was successfully catalogued and ready for analytical processing.

Athena Reporting Views

Eleven views were created on top of the visualization table to support reporting and to keep Power BI from querying the raw table directly.

1. Product Details View (vw_products)

```
CREATE OR REPLACE VIEW vw_products AS
SELECT
    review_year,
    main_category,
    sub_category,
    COUNT(DISTINCT parent_asin) AS total_products,
    AVG(product_average_rating) AS average_product_rating,
    AVG(product_rating_count) AS average_rating_count,
    MAX(product_rating_count) AS highest_rating_count
FROM visualization
GROUP BY
    review_year,
    main_category,
    sub_category;
```

2. Reviews View (vw_reviews)

```
CREATE OR REPLACE VIEW vw_reviews AS
SELECT
    review_year,
    main_category,
    rating_category,
    purchase_type,
    COUNT(*) AS total_reviews,
    AVG(review_rating) AS average_rating,
    SUM(helpful_vote) AS total_helpful_votes,
    AVG(review_length) AS average_review_length,
    AVG(review_word_count) AS average_word_count
FROM visualization
GROUP BY
    review_year,
    main_category,
    rating_category,
    purchase_type;
```

3. Time Analysis View (vw_time_analysis)

```
CREATE OR REPLACE VIEW vw_time_analysis AS
SELECT
    review_year,
    main_category,
    review_month,
    review_month_name,
    review_quarter,
    review_year_month,
    review_day_of_week,
    COUNT(*) AS total_reviews,
    AVG(review_rating) AS average_review_rating,
    SUM(helpful_vote) AS total_helpful_votes
FROM visualization
GROUP BY
    review_year,
    main_category,
    review_month,
    review_month_name,
    review_quarter,
    review_year_month,
    review_day_of_week;
```

4. Category Summary View (vw_category_summary)

```
CREATE OR REPLACE VIEW vw_category_summary AS
SELECT
    review_year,
    main_category,
    sub_category,
    COUNT(*) AS total_reviews,
    AVG(review_rating) AS average_review_rating,
    AVG(product_average_rating) AS average_product_rating,
    SUM(product_rating_count) AS total_product_ratings
FROM visualization
GROUP BY
    review_year,
    main_category,
    sub_category;
```

5. Purchase Analysis View (vw_purchase_analysis)

```
CREATE OR REPLACE VIEW vw_purchase_analysis AS
SELECT
    review_year,
    main_category,
    purchase_type,
    verified_purchase,
    COUNT(*) AS total_reviews,
    AVG(review_rating) AS average_rating,
    SUM(helpful_vote) AS total_helpful_votes
FROM visualization
GROUP BY
    review_year,
    main_category,
    purchase_type,
    verified_purchase;
```

6. Helpfulness Analysis View (vw_helpfulness)

```
CREATE OR REPLACE VIEW vw_helpfulness AS
SELECT
    review_year,
    main_category,
    helpful_vote_bucket,
    COUNT(*) AS total_reviews,
    SUM(helpful_vote) AS total_helpful_votes,
    AVG(review_rating) AS average_review_rating
FROM visualization
GROUP BY
    review_year,
    main_category,
    helpful_vote_bucket;
```

7. Product Performance View (vw_product_performance)

```
CREATE OR REPLACE VIEW vw_product_performance AS
SELECT
    review_year,
    main_category,
    sub_category,
    COUNT(DISTINCT parent_asin) AS total_products,
    COUNT(*) AS total_reviews,
    AVG(product_average_rating) AS average_product_rating,
    AVG(review_rating) AS average_review_rating,
    SUM(helpful_vote) AS total_helpful_votes,
    AVG(review_length) AS average_review_length,
    AVG(review_word_count) AS average_review_word_count
FROM visualization
GROUP BY
    review_year,
    main_category,
    sub_category;
```

8. Rating Distribution View (vw_rating_distribution)

```
CREATE OR REPLACE VIEW vw_rating_distribution AS
SELECT
    review_year,
    main_category,
    rating_category,
    COUNT(*) AS total_reviews
FROM visualization
GROUP BY
    review_year,
    main_category,
    rating_category;
```

9. Review Rating Summary (vw_review_rating_summary)

```
CREATE OR REPLACE VIEW vw_review_rating_summary AS
SELECT
    CAST(review_rating AS INTEGER) AS star_rating,
    COUNT(*) AS total_reviews
FROM visualization
GROUP BY CAST(review_rating AS INTEGER);
```

10. Category Dimension (dim_category)

```
CREATE OR REPLACE VIEW dim_category AS
SELECT DISTINCT
    main_category
FROM visualization;
```

11. Year Dimension (dim_year)

```
CREATE OR REPLACE VIEW dim_year AS
SELECT DISTINCT
    review_year
FROM visualization;
```


Step 8: Prepare the Data for Business Intelligence

With the data successfully available in Athena, the next phase of the project is to optimise it for reporting. The planned tasks include:

- Creating Athena Views for reporting.
- Applying partitioning wherever applicable to improve query performance.
- Connecting Power BI to Amazon Athena.
- Building interactive dashboards using the Athena Views instead of directly querying the raw table.

Dashboard Visualization

The Amazon Customer Review Analytics Dashboard was built in Power BI on top of the Athena views. It surfaces review volume, ratings, sentiment, and verification trends across the catalog.

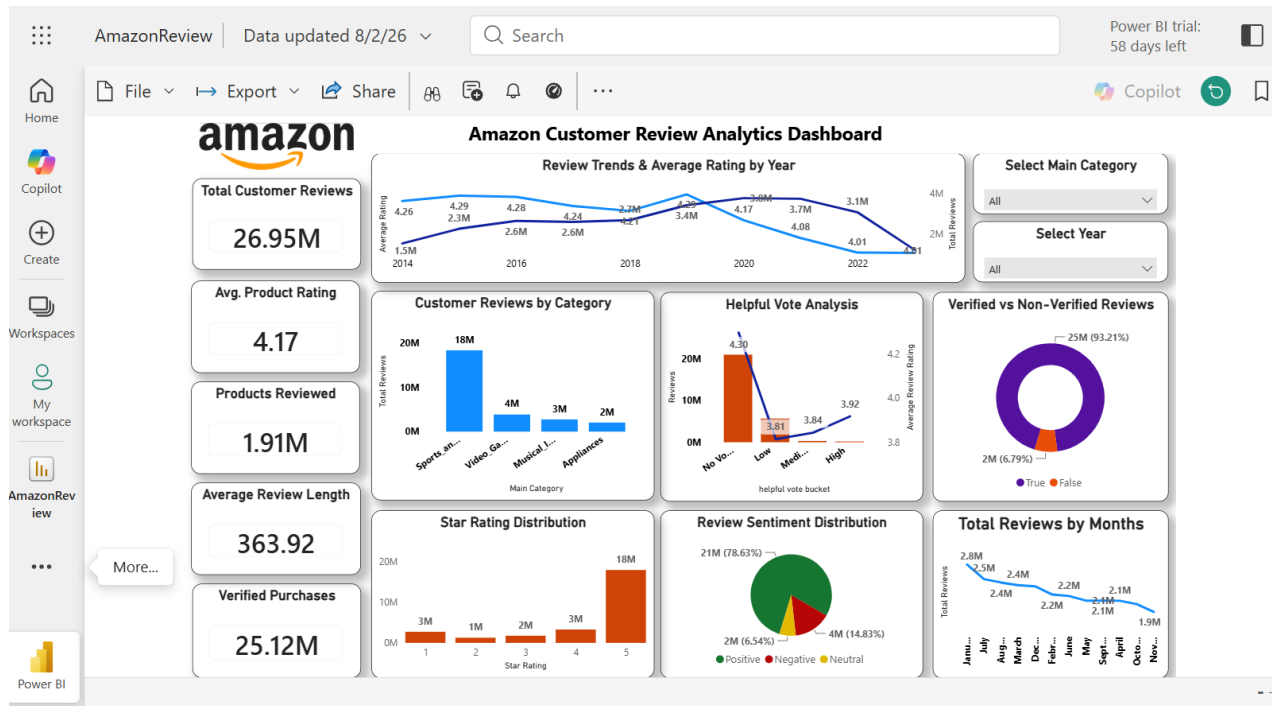


Figure 2. Amazon Customer Review Analytics Dashboard in Power BI.

Outcome

The project successfully established a serverless analytics pipeline without duplicating data. The AWS Glue Crawler automatically catalogued the Parquet dataset stored in the shared S3 bucket. AWS Glue Data Catalog stored the metadata, Amazon Athena enabled SQL-based querying, and the processed data is now ready for visualization in Power BI through Athena.

This architecture is scalable, cost-effective, and follows AWS best practices by separating data storage (Amazon S3), metadata management (AWS Glue Data Catalog), query processing (Amazon Athena), and business intelligence (Power BI).